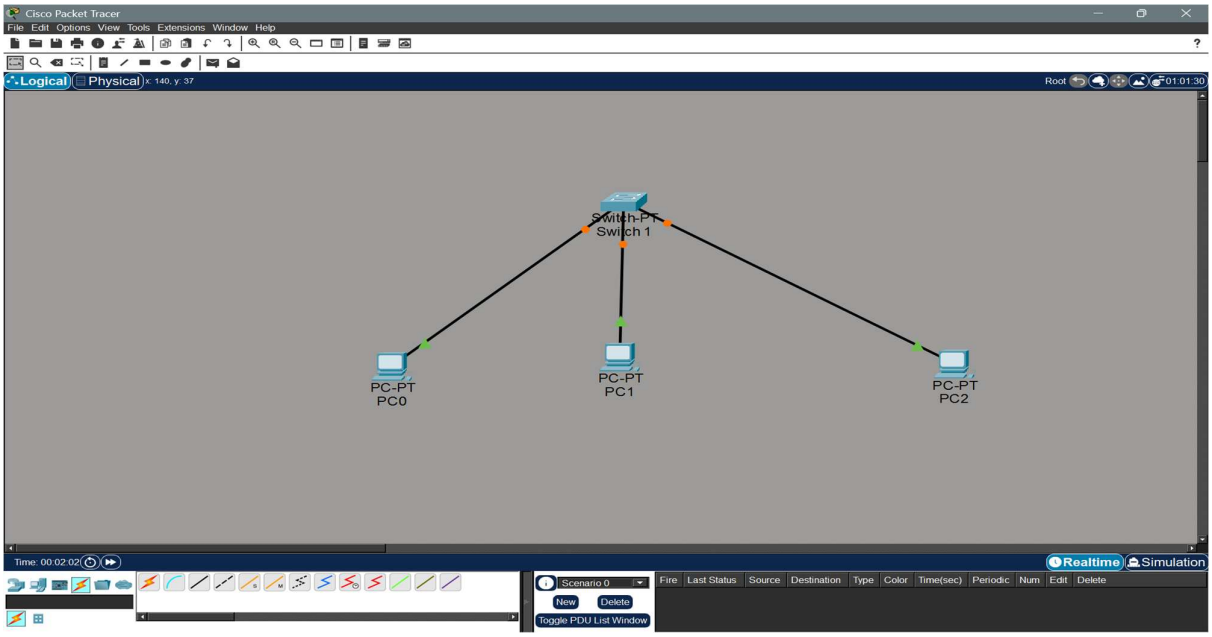
Structured Training Program: Networking

Week 2

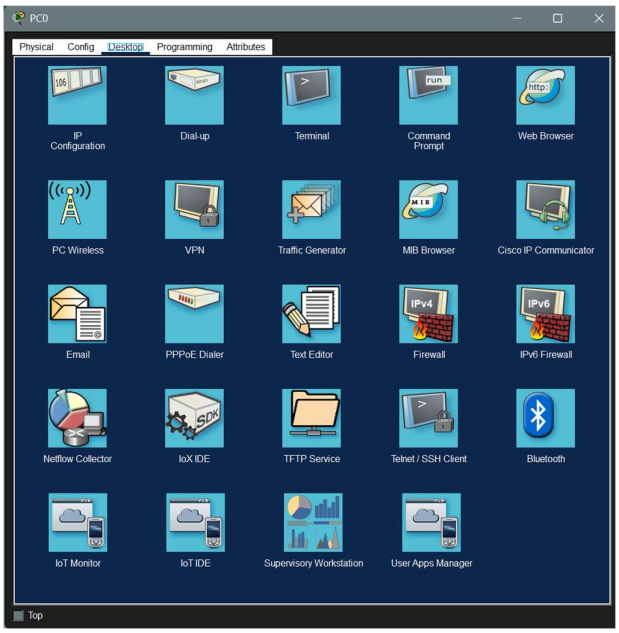
Module 3: Local Area Network – Layer 2 Protocols and Ethernet & Module 4: Switching

1. Simulate a small network with switches and multiple devices. Use ping to generate traffic and observe the MAC address table of the switch. Capture packets using wireshark to analyze Ethernet Frames and MAC addressing

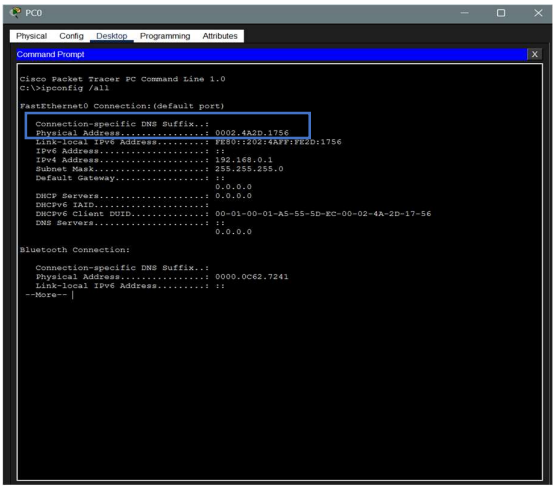
a) Take a switch (from networking devices list) and 3 end devices (I have taken 3 PCs) and connect them.



b) Configure IP in all the PCs.



c) Once configured IP on all 3 system, check the status of the initial CAM table in the switch before traffic. (If the switch already contains the MAC of PCS enter privilege mode and clear the table)



d) Now let me ping system PC0 to system PC1. Table will now take the src MAC from that packet and update the table.

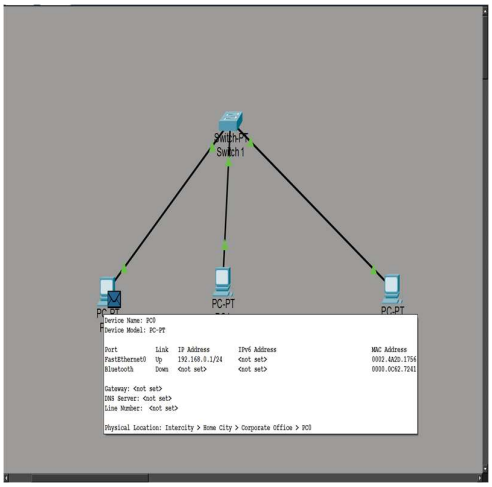


Fig 1. Simulation Starts

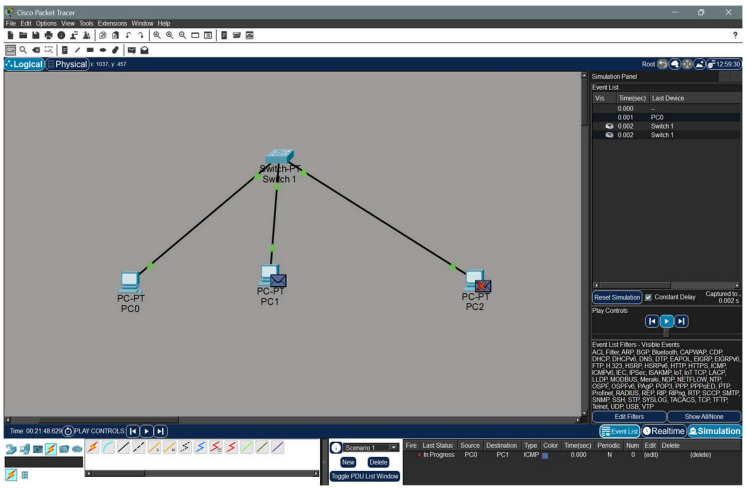


Fig 2. Broadcast all except the source

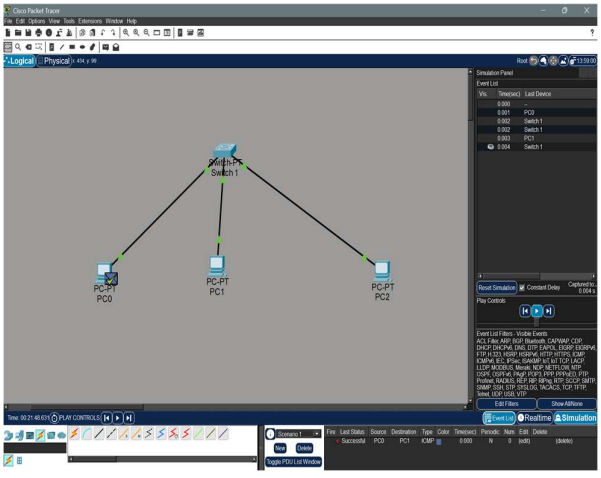


Fig 3. Final ACK sent to the sender

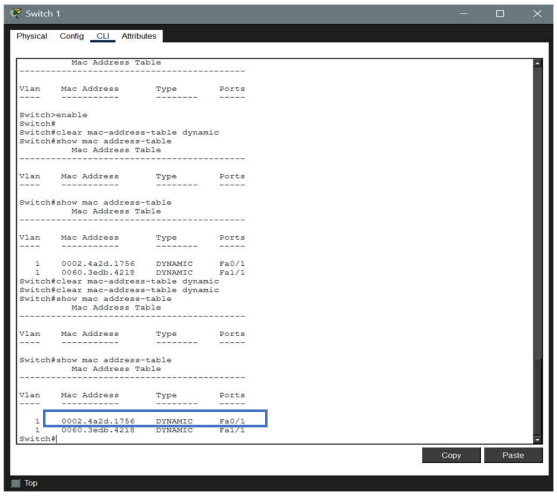
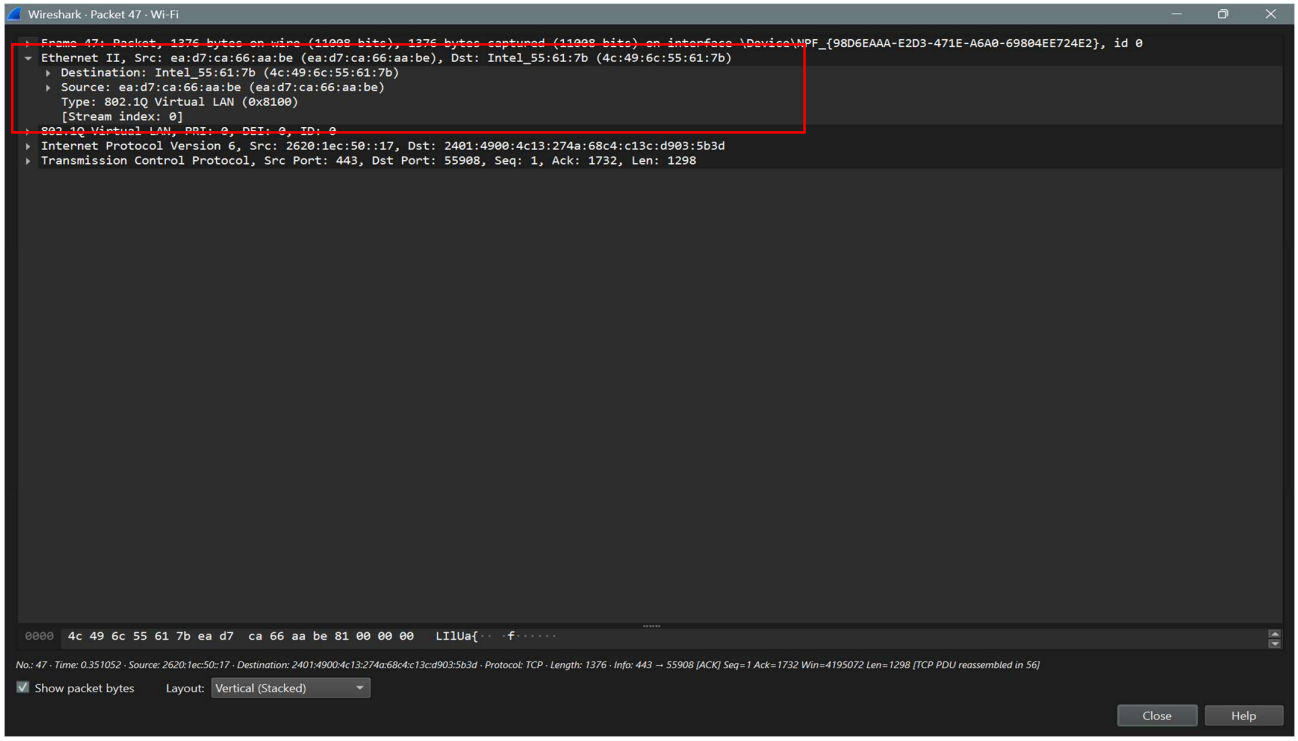


Fig 4. Current Status of the switch with MAC

e) We can see the source MAC address of the PC1 is present there. Since the flow contained an ACK packet sent to the sender the PC1's MAC address is also recorded in the table.

Analysing Ethernet frames using Wireshark:

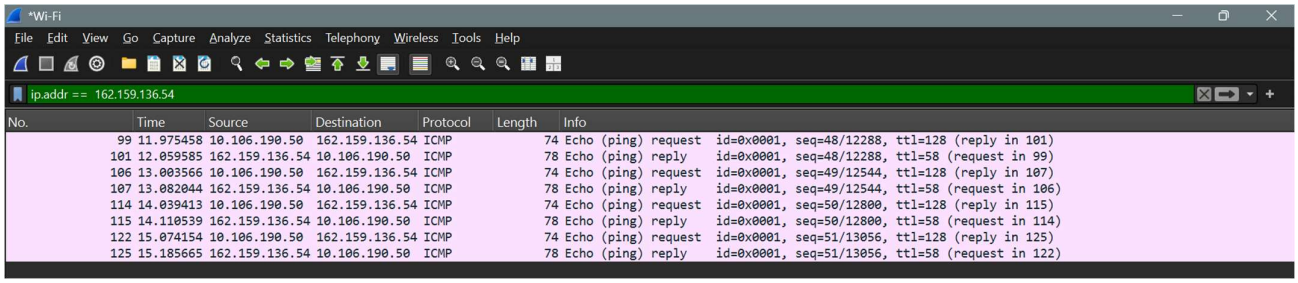


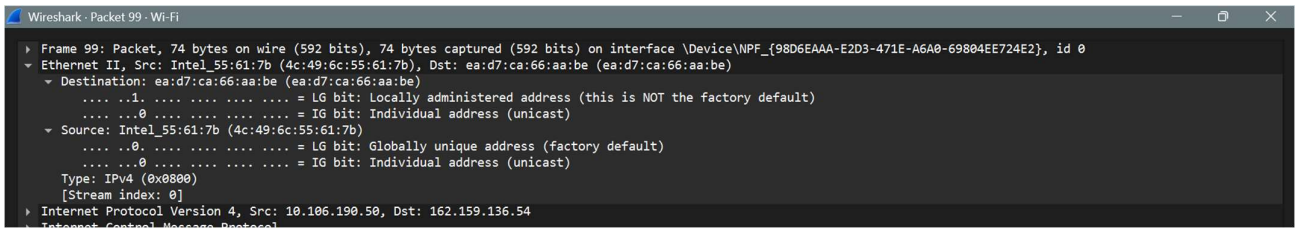
Observation:

a) The packet I have captured is a TCP packet sent to a random system service running on port 55908.
b) The src MAC address is ea:d7:ca:66:aa:be and the des MAC address is 4c:49:6c:55:61:7b (my WiFi card MAC Address)
c) From the OUI ID we can say the MAC address of dst belong to Intel.
d) The dst MAC address First 3 blocks don't seem to belong to any standard OUI. When the first block is expanded as binary, we get 11101010. So, we see that the last bit is 0 which says the MAC is a Unicast MAC ID and the previous bit is set to 1 which means the MAC is locally administered.
e) This confirms that the MAC address belongs to any server machine that provide service for any windows related background service.

2) Capture and analyse Ethernet frames using Wireshark. Inspect the structure of the frame, including destination and source MAC addresses, Ethernet Type, payload, and FCS. Use packet Tracer to simulate network traffic.

a) Using Wireshark to capture traffic. I have pinged the embedUR domain for analysis





Structure of the frame:

a) **Destination MAC address**: ea:d7:ca:66:aa:be
b) **Source MAC address**: 4c:49:6c:55:61:7b
c) **Ethernet Type**: IPV4
d) **Payload**: Represented as Hex + ASCII Dump

Entire Packet:

```
0000   ea d7 ca 66 aa be 4c 49 6c 55 61 7b 08 00 45 00   ...f..LIlUa{..E.
0010   00 3c 78 c9 00 00 80 01 00 00 0a 6a be 32 a2 9f   .<x........j.2..
0020   88 36 08 00 4d 2b 00 01 00 30 61 62 63 64 65 66   .6..M+...0abcdef
0030   67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76   ghijklmnopqrstuv
0040   77 61 62 63 64 65 66 67 68 69                     wabcdefghi
```

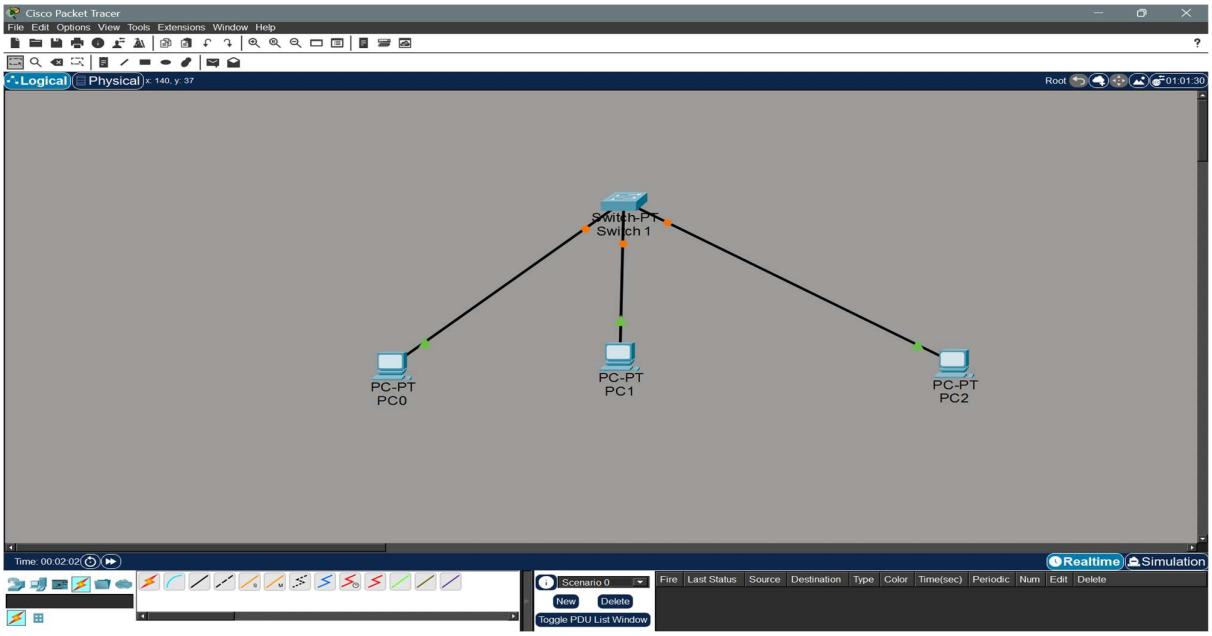
Ethernet Frame Only:

```
0000   ea d7 ca 66 aa be 4c 49 6c 55 61 7b 08 00         ...f..LIlUa{..
```

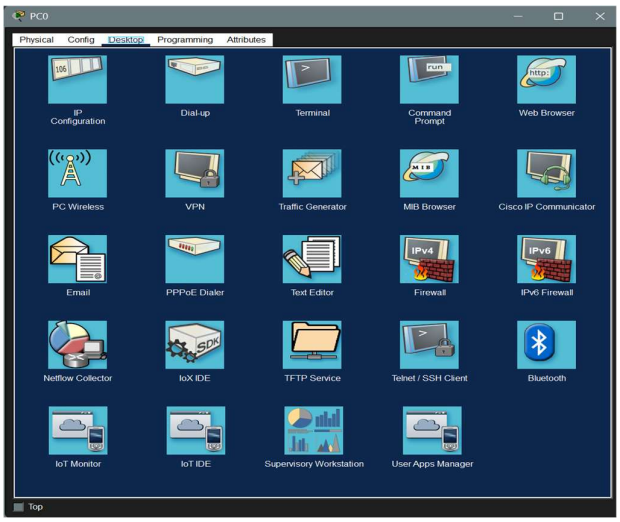
Data Only:

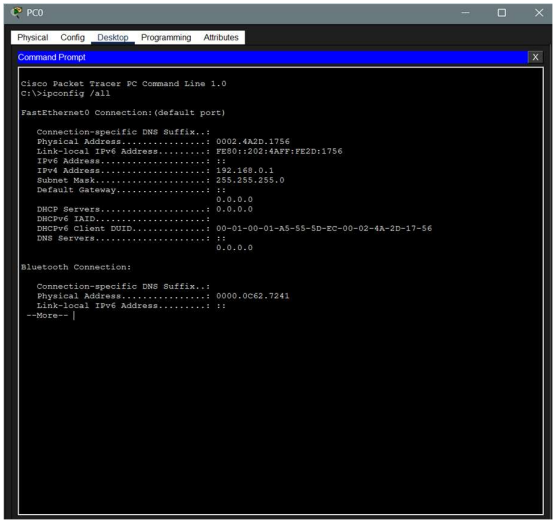
```
0000   61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70   abcdefghijklmnop
0010   71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69   qrstuvwabcdefghi
```

Simulating Network Traffic with packet Tracer:

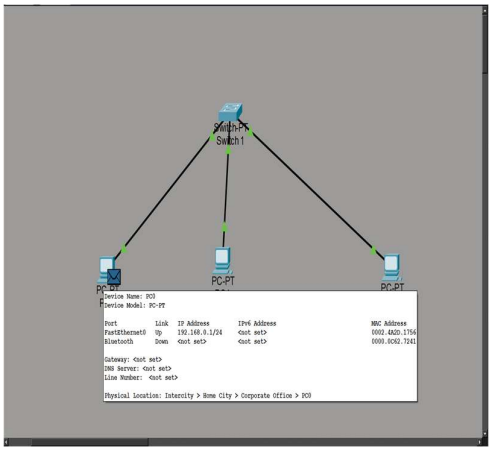


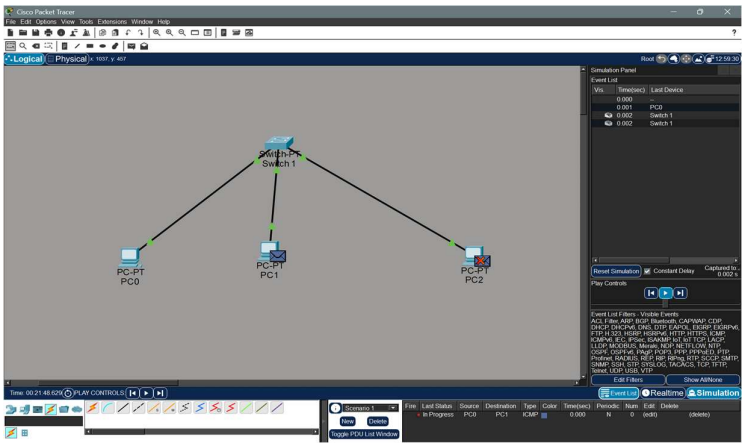
Connect the systems





Configure the IP address

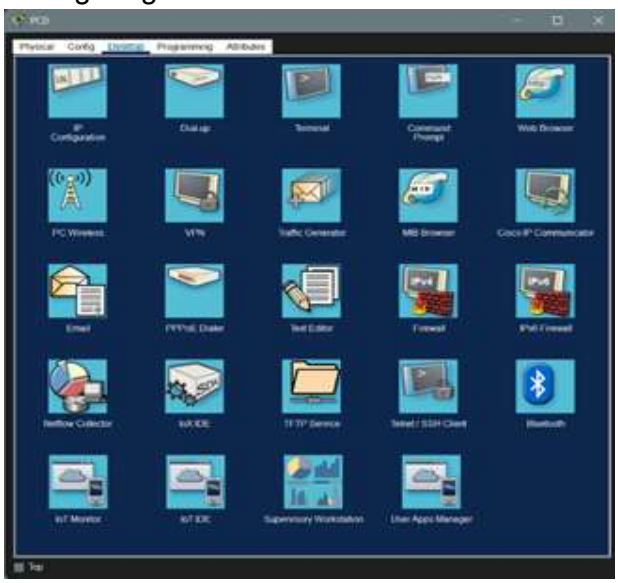




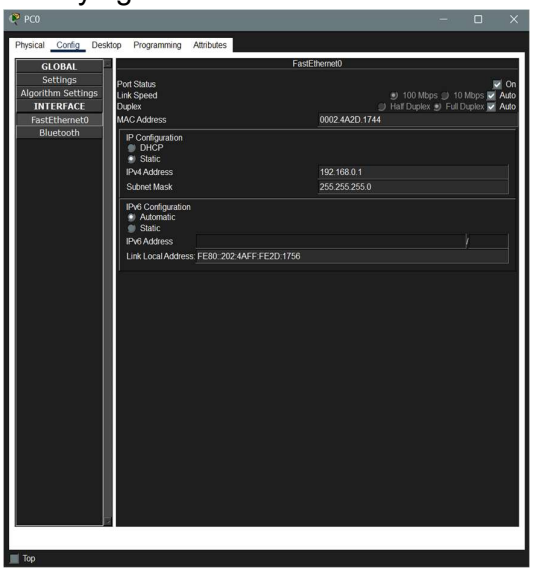
Select system to share packets and start simulation

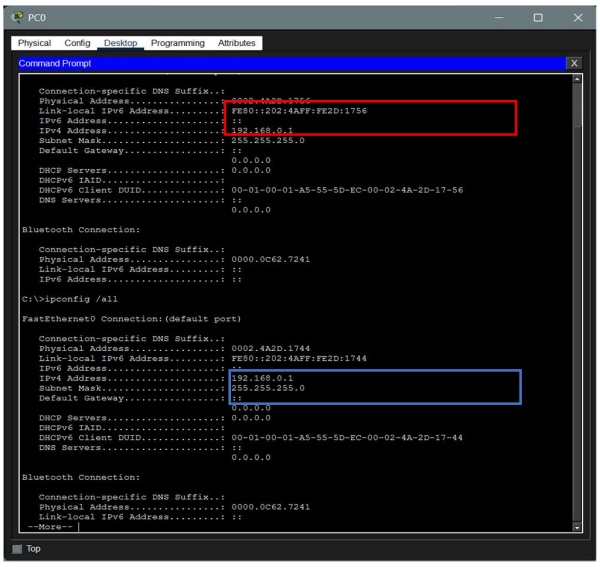
3) Configure Static IP address, modify MAC address, and verify network connectivity using ping and ifconfig.

a) Configuring static IP address

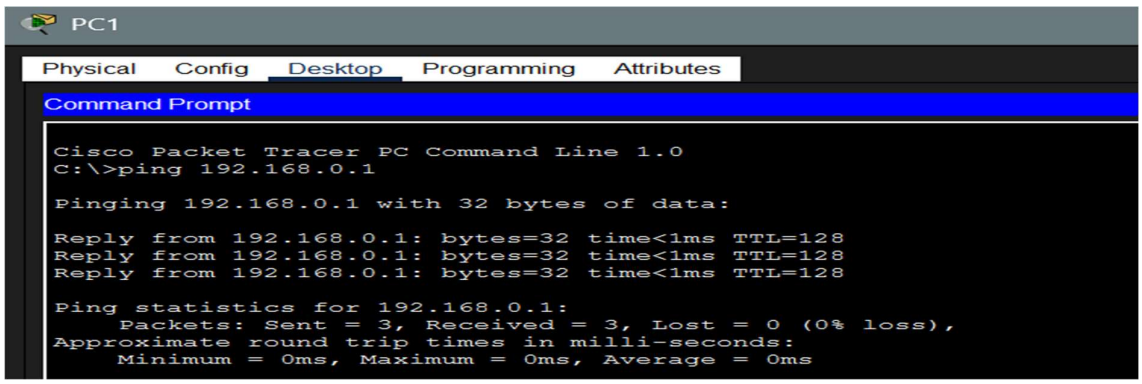


b) Modifying MAC address



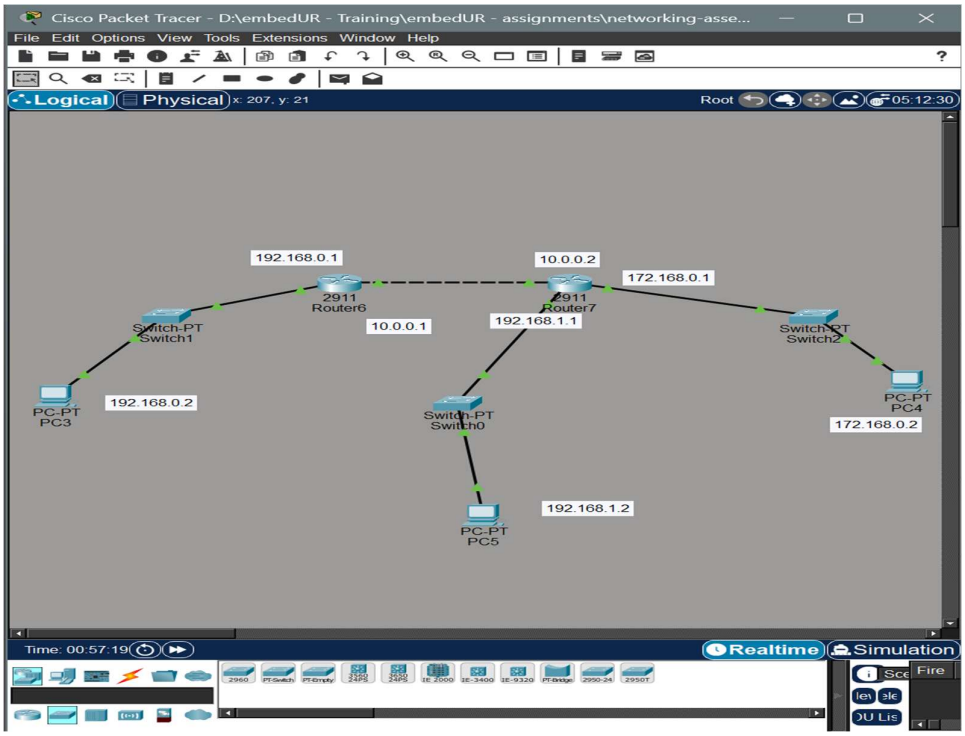


c) Verifying network connectivity using ping and ifconfig (ping pc0 from pc1)

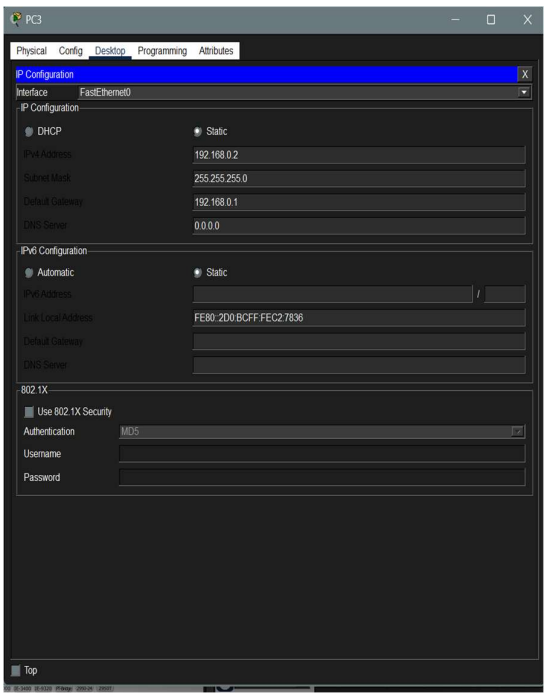


4. Troubleshoot Ethernet communication with ping and tracert. Use Cisco packet Tracer

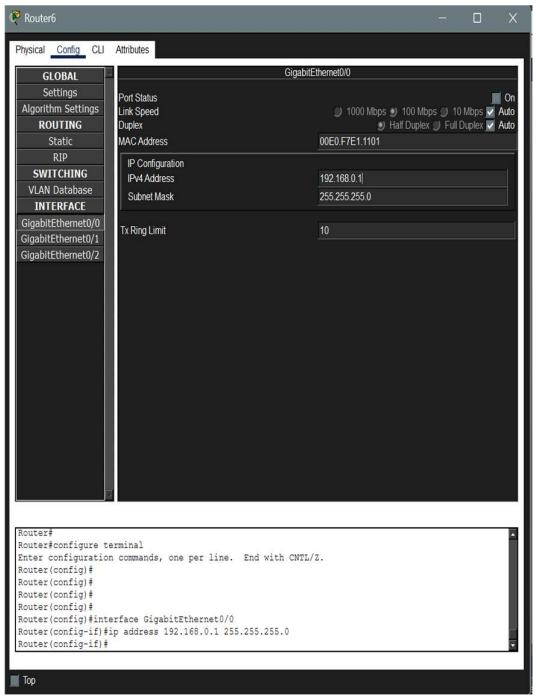
a) Create a network topology with atleast 2 or 3 different subnets to show the connectivity related problems



b) In each subnet system configure the IP, subnet and the gateway IP properly. The gateway IP given in PC and router must be same. Repeat this for all subnet

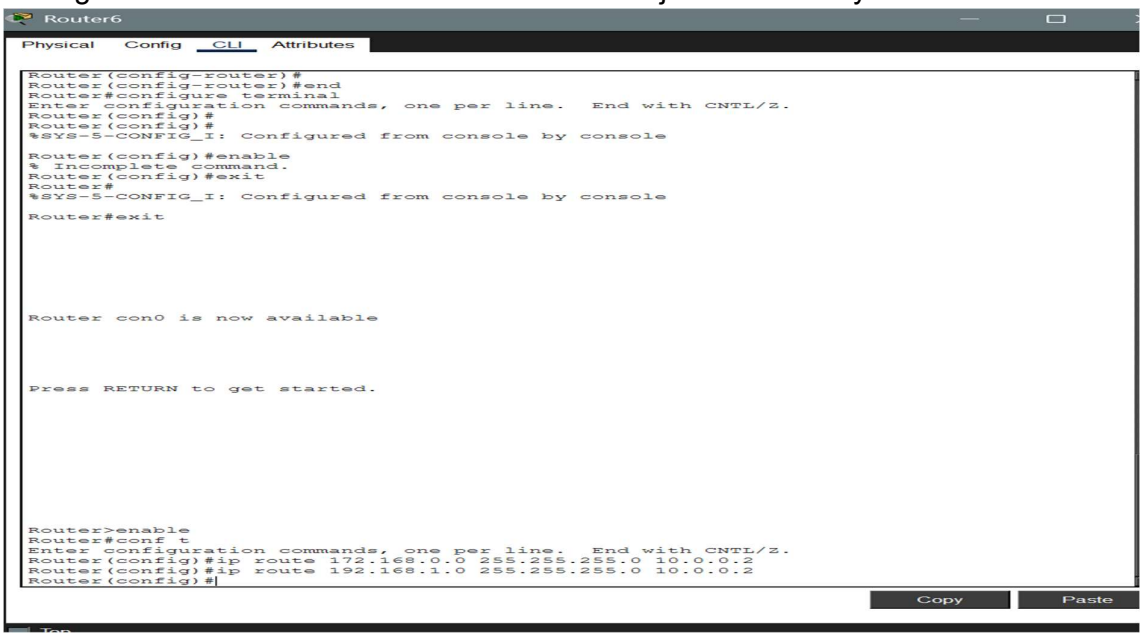


PC Configuration

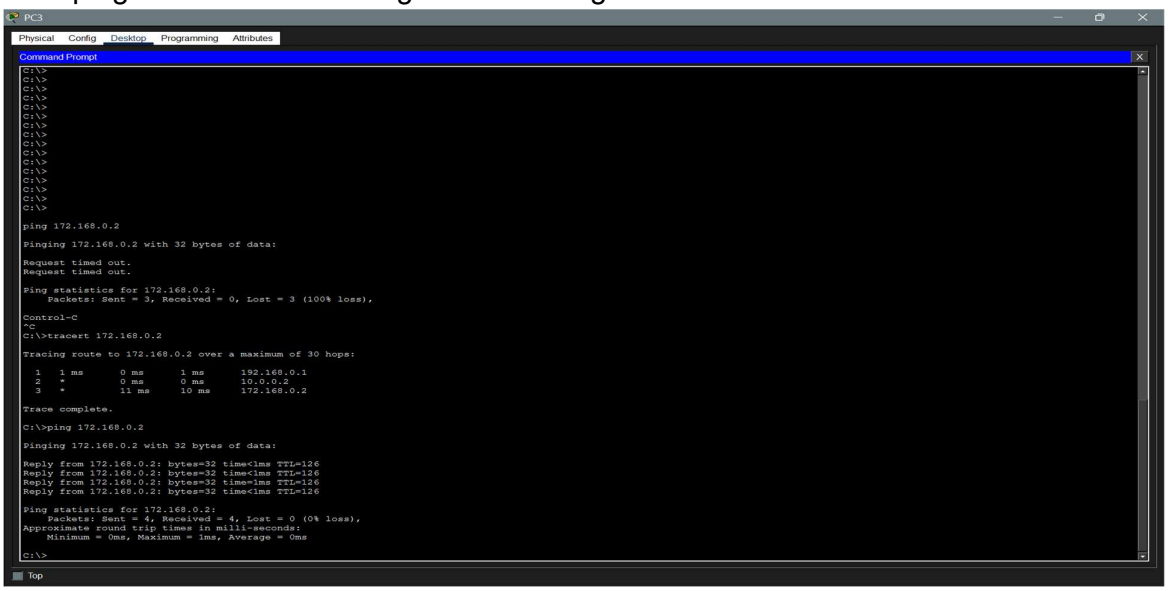


Router Configuration

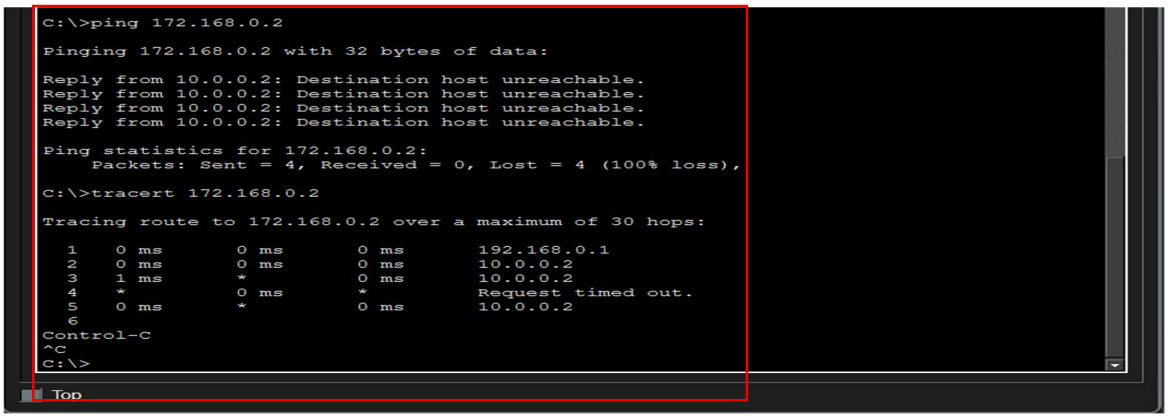
d) Configure the router with the IP address of the adjacent Gateways



e) Now ping the device the configure the routing table



f) Now let's off one interface such as the in Gig (0/2) in router 7 and debug with tracert

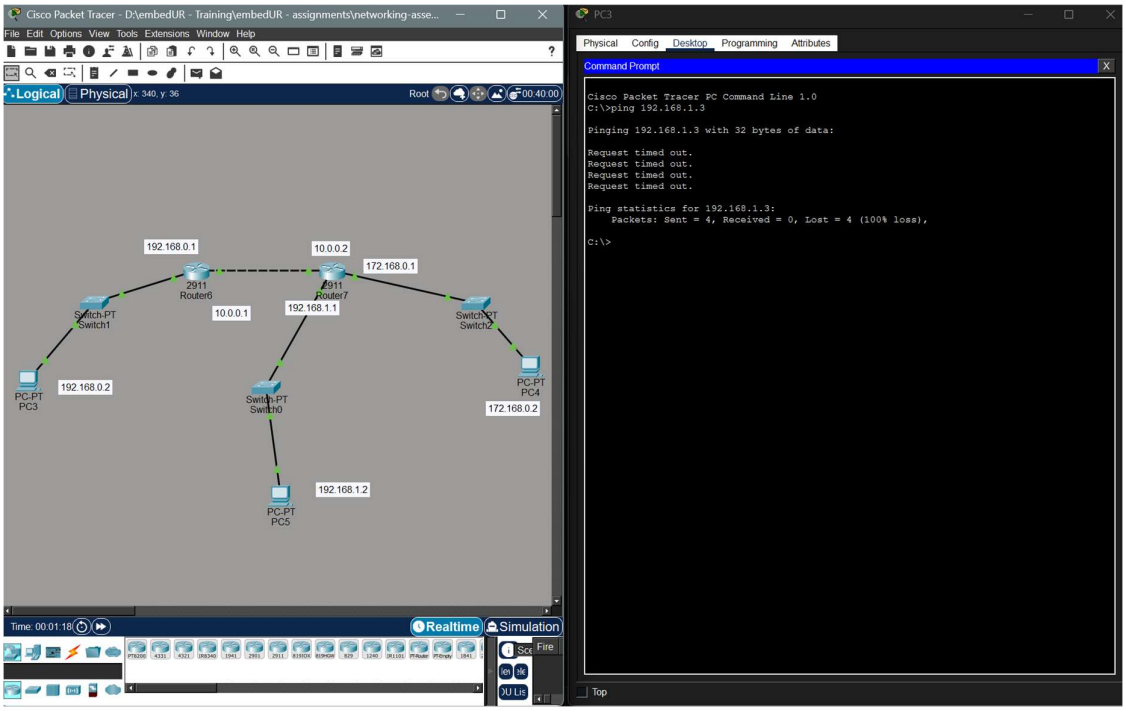


We can clearly see that there is some problem after 10.0.0.2, using this we can go to that port and solve the issue at that router.

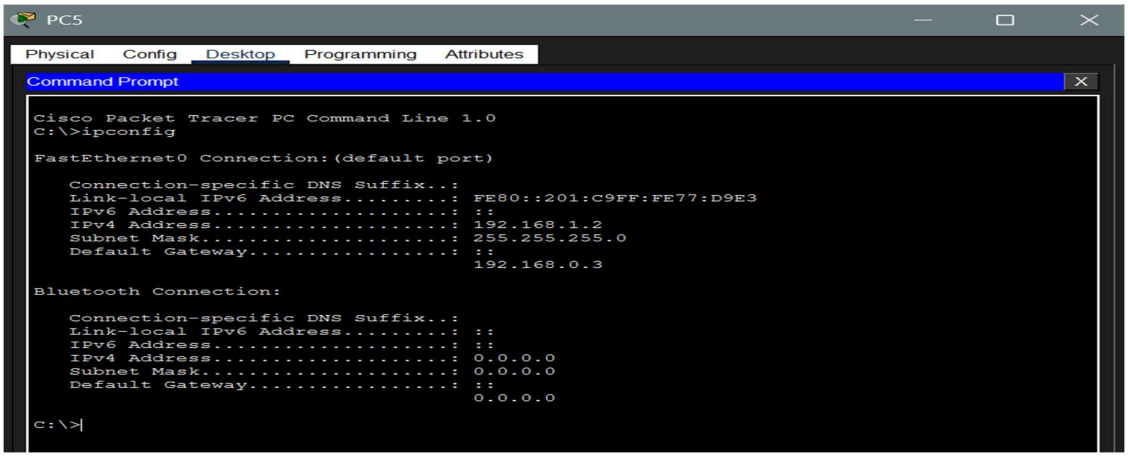
5. Create a simple LAN setup with two Linux machines connected via switch.(done as last qn)

6. Ping from one system to another if the ping fails use ifconfig to ensure that the IP addresses are configured correctly.

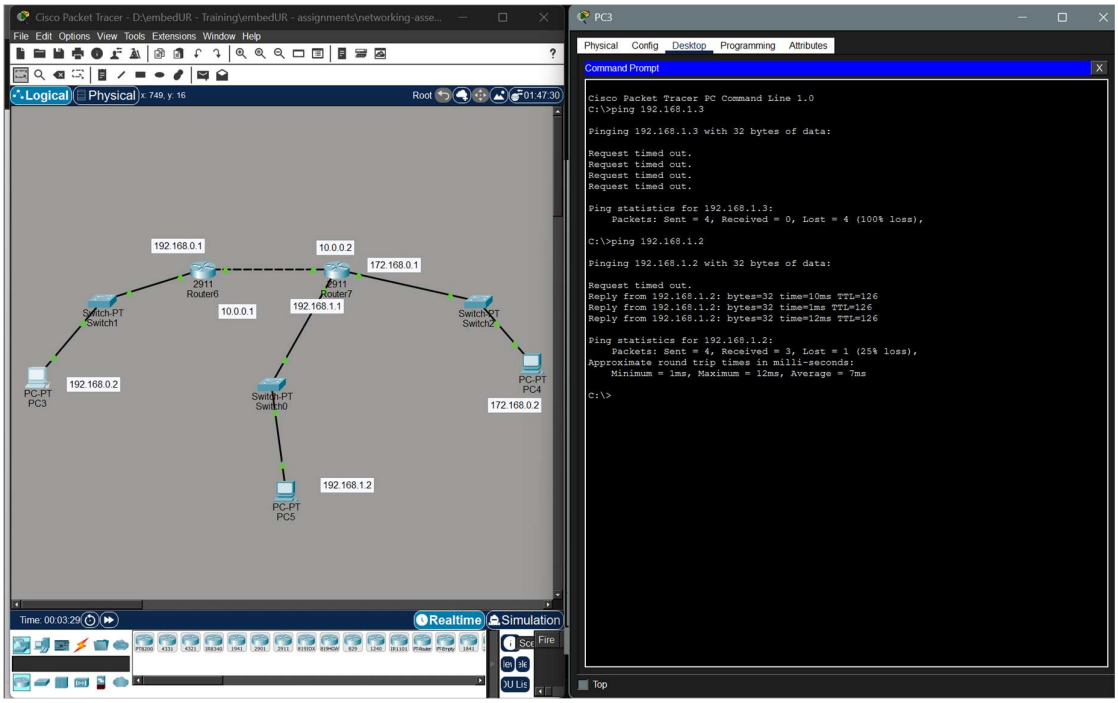
a) Using ping to check if two systems are connected.



b) Using ifconfig to check if the system IP is configured correctly

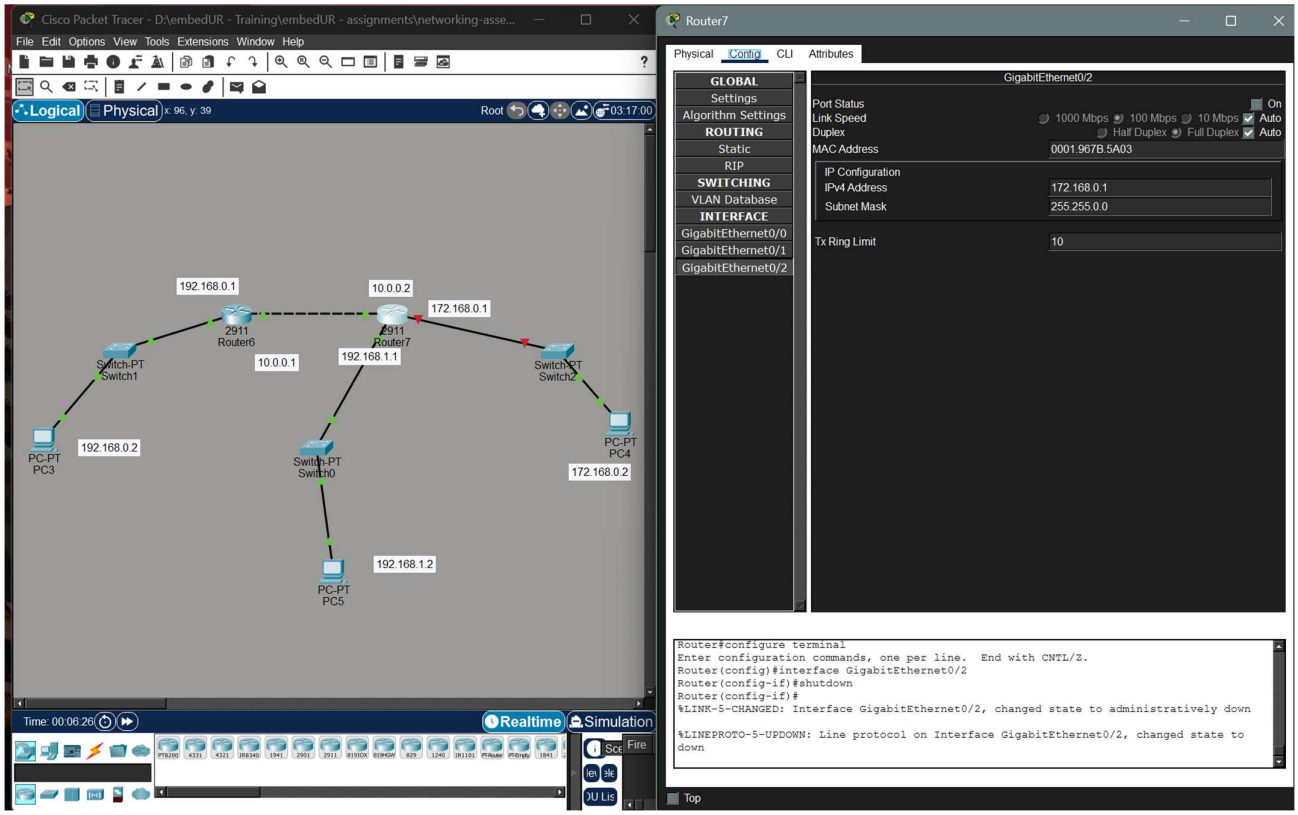


c) We have pinged with a wrong IP now we can correct the IP address in the ping command

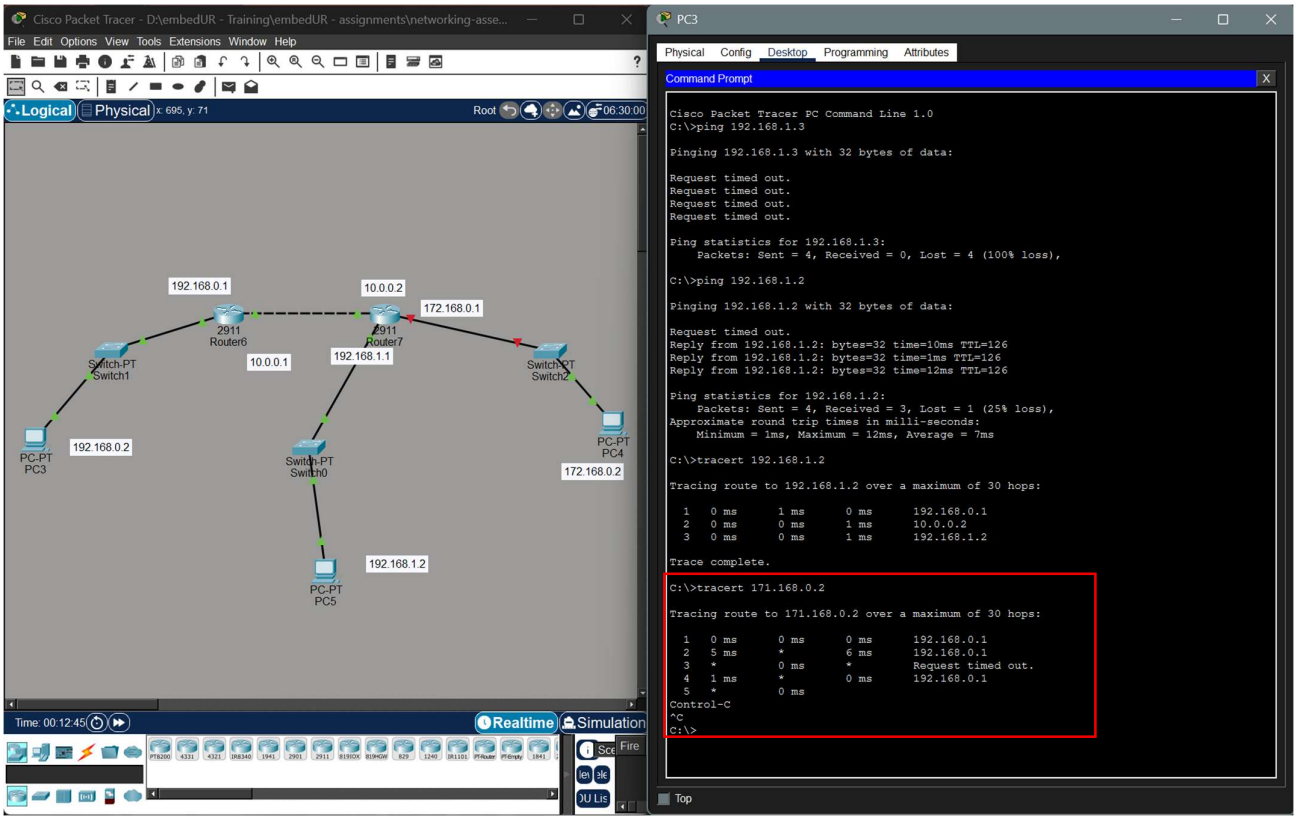


7. Use traceroute to find where the packet are being dropped if the ping fails

a) I have turned off one the interface of router (Gig 0/2) of router 7 to test this. The IP address of this interface is 172.



b. Now I am ping the subnet for which the router 7 interface Gig 0/2 acts as the Gateway



We can see there is a clear drop in packet after router 6, this shows there is an issue in next hop. Using which we can confirm which router is faulty.

8. Research the Linux kernel's handling of the Ethernet devices and network interface. Write a short report on how the Linux kernel supports Ethernet communication.

Documents Referred:

Linux Source Code: https://github.com/torvalds/linux/tree/master/net

Linux Documentation: https://docs.kernel.org/networking/netdevices.html

The Linux kernel provides Ethernet communication support through its networking subsystem and device driver architecture. Ethernet devices are managed as network interfaces and integrated into the Linux network stack, allowing user applications to send and receive data over a network. The kernel handles packet processing, hardware interaction, and protocol management through several core data structures and subsystems.

Linux Networking Stack Architecture

The Linux networking stack is organized in layers that correspond to the OSI model. Applications communicate with the kernel through the socket interface implemented using the struct socket and struct sock data structures. Data flows through multiple protocol layers inside the kernel:

- Application Layer – User applications interact through system calls like send() and recv().
- Transport Layer – Implemented using structures such as struct tcp_sock for TCP and struct udp_sock for UDP.

- Network Layer – Uses struct iphdr to represent IP headers.
- Data Link Layer (Ethernet) – Managed using Ethernet frame structures like struct ethhdr.

Packets are represented inside the kernel using the socket buffer structure, struct sk_buff. This structure stores packet data, protocol headers, and metadata as packets travel through the networking stack.

Ethernet Device Drivers

Ethernet devices are supported in Linux using network device drivers. Each driver represents a network interface and registers it with the kernel using the structure, struct net_device

This structure contains information about the device such as:

- MAC address
- device name (e.g., eth0)
- transmit and receive queues
- operational flags

The driver defines device operations using, struct net_device_ops

Important functions inside this structure include:

- ndo_open() – Starts the network device
- ndo_stop() – Stops the device
- ndo_start_xmit() – Handles packet transmission
- ndo_set_rx_mode() – Configures receive mode

These operations allow the kernel to control the Ethernet device and send packets through the network interface card (NIC).

Packet Transmission

When an application sends data, the kernel creates a packet stored in a struct sk_buff. The packet moves through the networking stack and eventually reaches the Ethernet driver.

The driver uses the function:

ndo_start_xmit(struct sk_buff *skb, struct net_device *dev)

to transmit the packet. The packet is then placed into the Transmit Queue (TX queue) and sent to the NIC hardware.

The Ethernet header is constructed using, struct ethhdr

which includes fields such as:

- Destination MAC address
- Source MAC address
- EtherType field

Packet Reception

When an Ethernet frame arrives at the network interface card, the NIC triggers a hardware interrupt. The Linux kernel processes these using mechanisms such as NAPI (New API) for efficient packet handling.

The driver retrieves the received packet and stores it in a struct sk_buff. The packet is then passed to the networking stack using functions like, netif_receive_skb()

The kernel analyses the Ethernet header (struct ethhdr) and forwards the packet to the appropriate protocol layer (IP, TCP, UDP) until it reaches the destination application.

Network Interface Representation

In Linux, Ethernet interfaces appear as devices such as eth0, eth1, or modern names like enp3s0. These interfaces are managed by the kernel through the net_device subsystem.

Configuration and management of these interfaces can be performed using tools such as:

- ip command
- ifconfig
- ethtool

These tools communicate with the kernel using netlink sockets (NETLINK_ROUTE) to configure network interfaces.

9. Describe how would you configure a basic LAN interface using the ip command in Linux Kernel.

Identify the network interface:

a) To list all available network interfaces:

   sudo ip link show

b) Bring the interface down:

   sudo ip link set dev eth0 down

c) Assign an IP address and subnet mask:

   sudo ip addr add 192.168.1.100/24 dev eth0

   (Note: Add the static IP address and subnet mask (using CIDR notation) to the interface)

d) Bring the interface up:

   sudo ip link set dev eth0 up

e) Set the default gateway:

sudo ip route add default via 192.168.1.1 dev eth0

f) Remove an IP address: **sudo ip addr del 192.168.1.1/24 dev eth0**

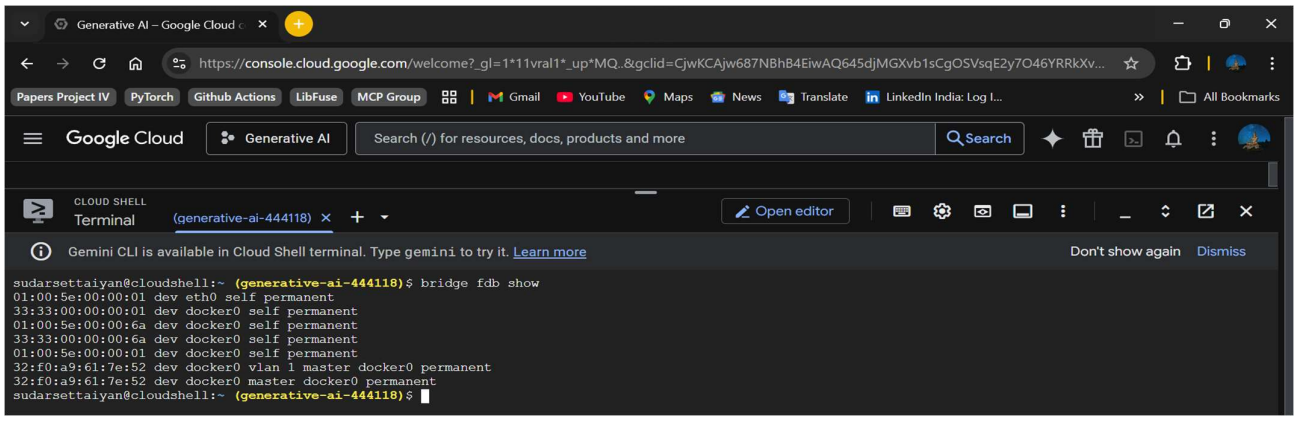
g) Flush all IP addresses: **sudo ip addr flush eth0**

h) View routing table: **sudo ip route show**

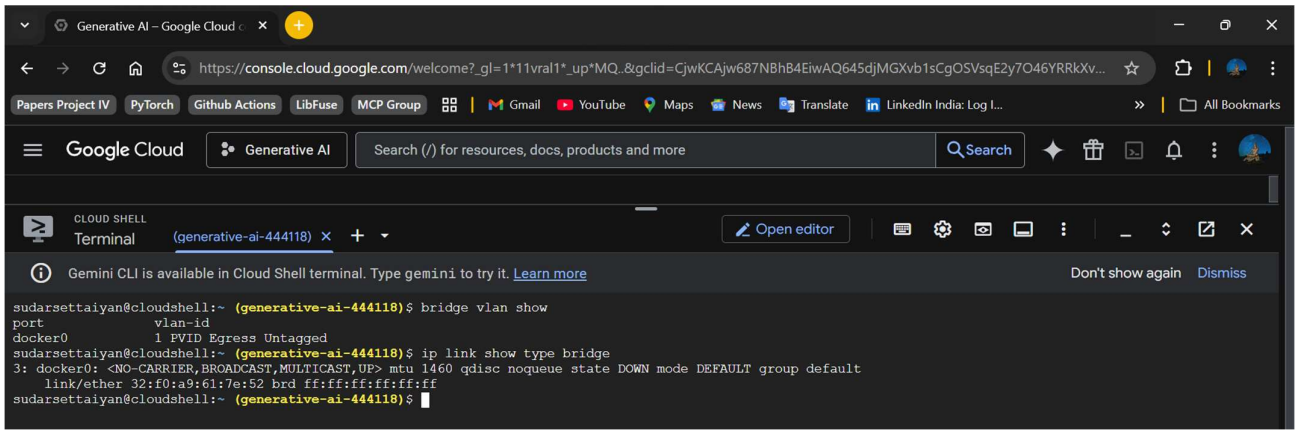
i) Verify the configuration: **sudo ip addr show eth0**

10. Use Linux to view the MAC address table of the switch (if using a Linux based network switch). Use the bridge or ip link command to inspect the MAC table and demonstrate a basic switch operation

a) View the MAC address table of a switch (equivalent to show mac address-table in cisco switch)



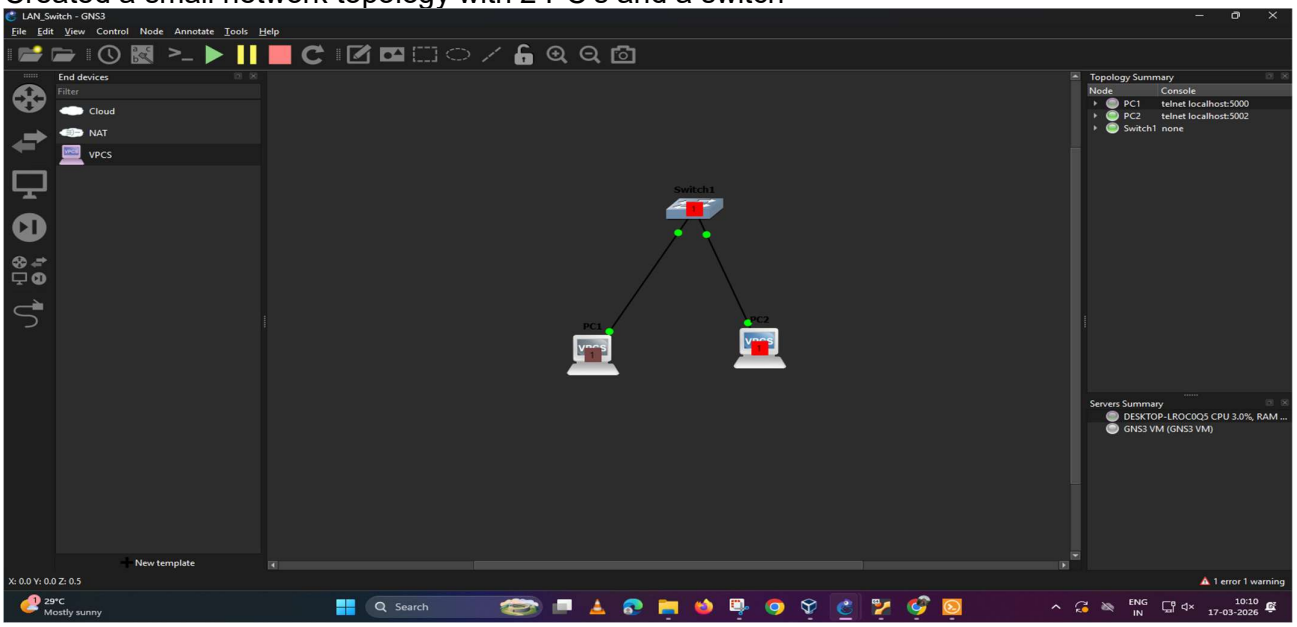
b) Using bridge or ip link command to see / inspect the bridge



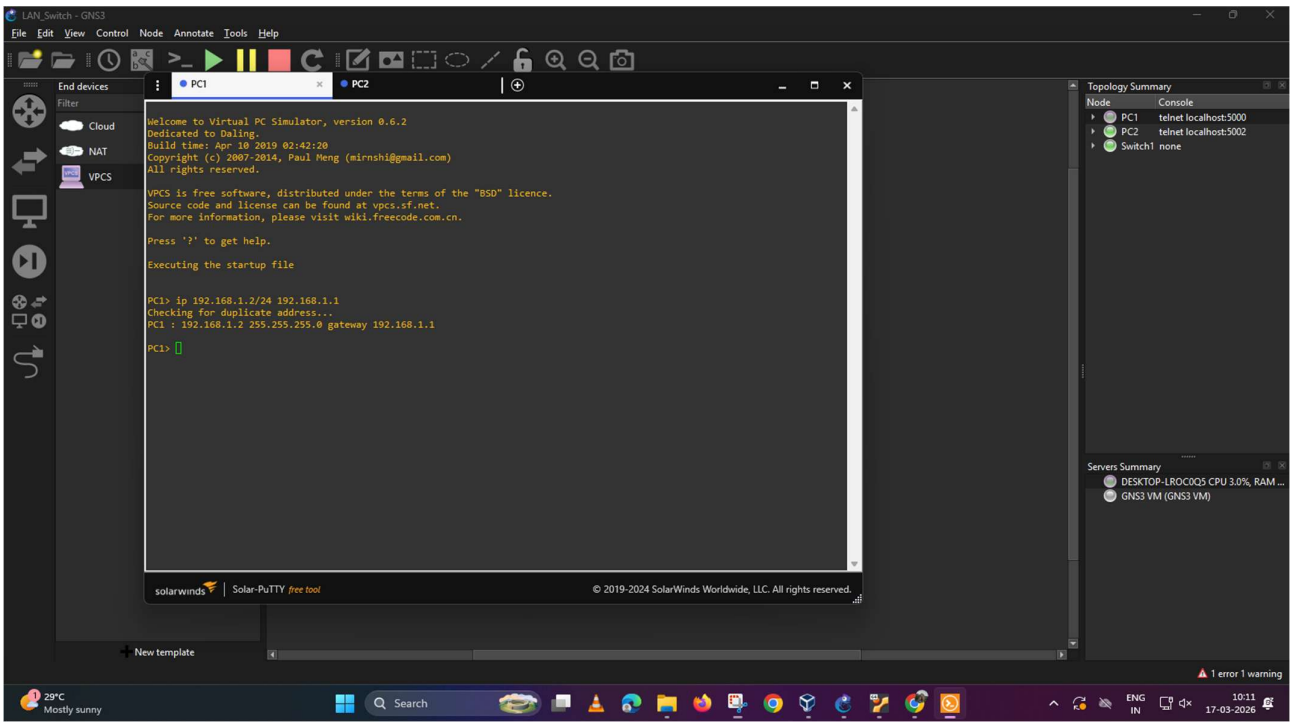
5. Create a simple LAN network with 2 Linux machine connected via a switch

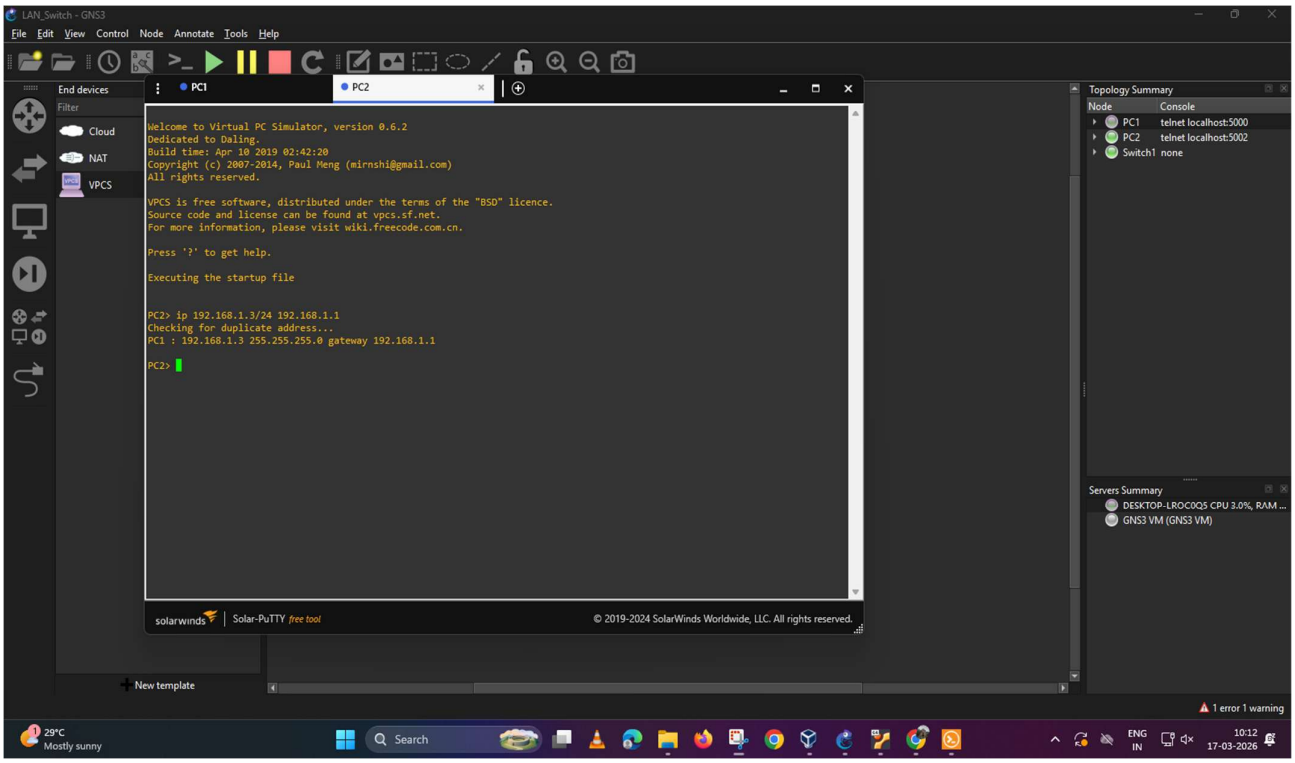
I have used GNS3 software for this simulation.

Created a small network topology with 2 PC’s and a switch



Assign IPs to each PC





Ping one system from the other